Customer Shopping Behavior Analysis

1. Project Overview

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

2.Dataset Summary

- Rows: 3,900
- Columns: 18
- Key Features :-
- Customer demographics (Age, Gender, Location, Subscription Status)
- Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
- Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

- Data Loading: Imported the dataset using pandas.
- Initial Exploration: Used df.info() to check structure and .describe() for summary statistics.

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900	3900.000000	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2	NaN	6	7
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No	NaN	PayPal	Every 3 Months
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223	NaN	677	584
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN	25.351538	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN	14.447125	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN	1.000000	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN	13.000000	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN	25.000000	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN	38.000000	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN	50.000000	NaN	NaN

Missing Data Handling: Checked for null values and imputed missing values in the Review Rating column using the median rating of each product category.

● **Column Standardization:** Renamed columns to **snake case** for better readability and documentation.

● Feature Engineering:

○ Created **age_group** column by binning customer ages.

○ Created **purchase_frequency_days** column from purchase data.

● **Data Consistency Check:** Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

● **Database Integration:** Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis.

4. Data Analysis using SQL (Business Transactions)

We performed structured analysis in PostgreSQL to answer key business questions:

1.Revenue by Gender – Compared total revenue generated by male vs female customers.

```sql
--Q1. What is the total revenue generated by male vs. female customers?
Select gender,SUM (purchase_amount) as revenue
from customer
group by gender
```

	gender text	revenue numeric
1	Female	75191
2	Male	157890

2.**High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

```sql
--Q2. Which Customer used a discount but still spent more than the average purchase amount?
Select Customer_id, purchase_amount
from customer
where discount_applied = 'Yes' and purchase_amount >= (select AVG(purchase_amount)from customer)
```

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62
11	24	88
12	29	94
13	32	79
14	33	67

Total rows: 839 Query complete 00:00:00.128

3. **Top 10 Products by Rating** – Found products with the highest average review ratings.

```
--Q3. Which are the top 10 products with the highest average review rating?
Select item_purchased,round(avg(review_rating::numeric),2) as "Average Product Rating"
from customer
group by item_purchased
order by avg(review_rating) desc
limit 10
```

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78
6	T-shirt	3.78
7	Handbag	3.78
8	Belt	3.76
9	Jacket	3.76
10	Sweater	3.76

4. Shipping Type Comparison – Compared average purchase amounts between Standard and Express shipping.

```
--Q4.Compare the average Purchase Amount Between Standard and Express Shipping?

select shipping_type,
ROUND(AVG(purchase_amount),2)
from customer
where shipping_type in ('Standard','Express')
group by shipping_type;
```

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

5. Subscribers vs. Non-Subscribers – Compared average spend and total revenue across subscription status.

```
--Q5. Do subscribed customers spend more? Compare average spend and total revenue
--between subscribers and non-subscribers.
SELECT subscription_status,
       COUNT(customer_id) AS total_customers,
       ROUND(AVG(purchase_amount),2) AS avg_spend,
       ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;
```

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

6. **Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

```sql
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT
    ITEM_PURCHASED,
    ROUND(
        100.0 * SUM(
            CASE
                WHEN DISCOUNT_APPLIED = 'Yes' THEN 1
                ELSE 0
            END
        ) / COUNT(*),
        2
    ) AS DISCOUNT_RATE
FROM
    CUSTOMER
GROUP BY
    ITEM_PURCHASED
ORDER BY
    DISCOUNT_RATE DESC
LIMIT
    5;
```

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.66
3	Coat	49.07
4	Sweater	48.17
5	Pants	47.37

7. **Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

```sql
--Q7. Segment customers into New, Returning, and Loyal based on their total
-- number of previous purchases, and show the count of each segment.
with customer_type as (
SELECT customer_id,previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN 'NEW'
    WHEN previous_purchases BETWEEN 2 AND 10 then 'Returning'
    ELSE 'Loyal'
    END AS customer_segment
    FROM customer
)

SELECT customer_segment,count(*) As " Number of customers"
from customer_type
group by customer_segment;
```

	customer_segment text	Number of customers bigint
1	Loyal	3116
2	NEW	83
3	Returning	701

Top 3 Products per Category – Listed the most purchased products within each category.

```
--Q8. What are the top 3 most purchased products within each category?

SELECT *
FROM
(SELECT
category,
        item_purchased AS product_name,
        COUNT(*) AS total_purchased,

ROW_NUMBER() OVER(
    PARTITION BY category
    ORDER BY COUNT(*) DESC
) AS rank
FROM customer
GROUP BY category , product_name)

WHERE rank <= 3;
```

	category text	product_name text	total_purchased bigint	rank bigint
1	Accessories	Jewelry	171	1
2	Accessories	Sunglasses	161	2
3	Accessories	Belt	161	3
4	Clothing	Blouse	171	1
5	Clothing	Pants	171	2
6	Clothing	Shirt	169	3
7	Footwear	Sandals	160	1
8	Footwear	Shoes	150	2
9	Footwear	Sneakers	145	3
10	Outerwear	Jacket	163	1
11	Outerwear	Coat	161	2

9.Repeat Buyers & Subscriptions – Checked whether customers with >5 purchases are more likely to subscribe.

```sql
--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

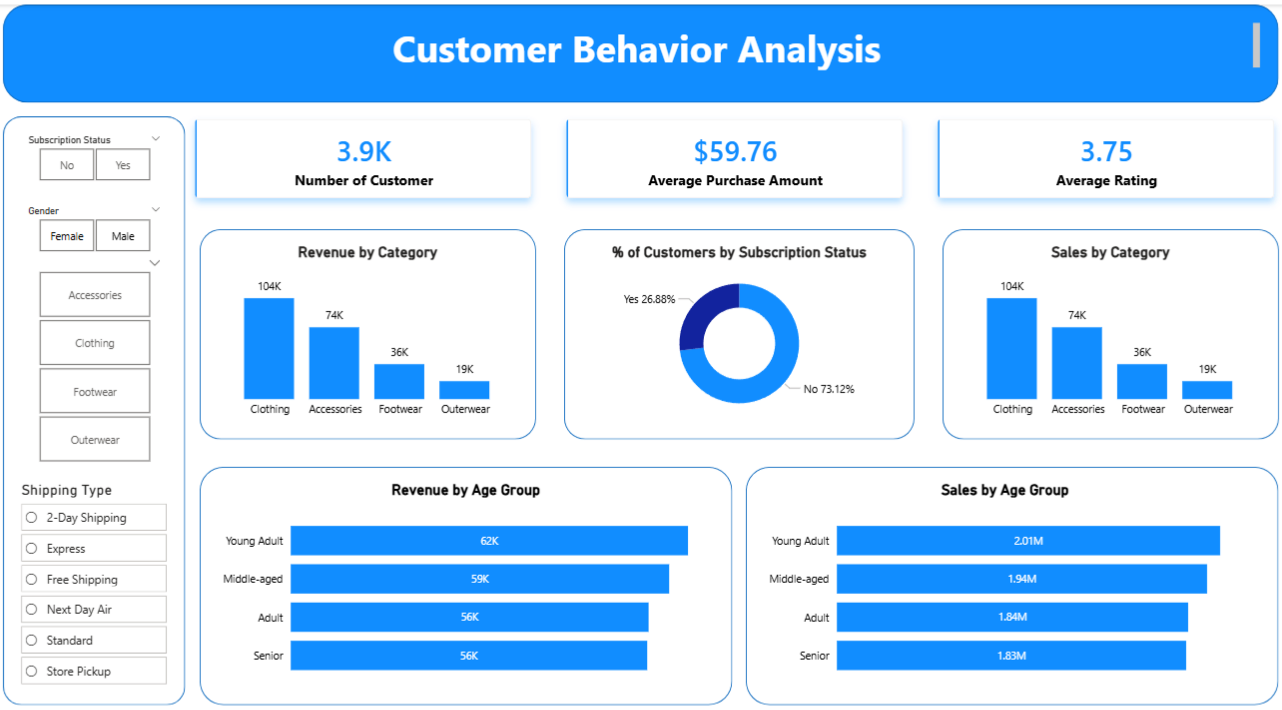
10. **Revenue by Age Group** – Calculated total revenue contribution of each age group

```sql
--Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY age_group
ORDER BY total_revenue desc;
```

	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

5. Dashboard in Power BI

Finally, we built an interactive dashboard in Power BI to present insights visually.



6. Business Recommendations

● Boost Subscriptions – Promote exclusive benefits for subscribers.

● Customer Loyalty Programs – Reward repeat buyers to move them into the "Loyal" segment.

● Review Discount Policy – Balance sales boosts with margin control.

● Product Positioning – Highlight top-rated and best-selling products in campaigns.

● Targeted Marketing – Focus efforts on high-revenue age groups and express-shipping users.